

Reviewer Pack — Minimal Exponent of Quaternion Algebras Bound Communication ...

Entry 1a7e97ca7d97 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-29 02:34:02 UTC

Minimal Exponent of Quaternion Algebras Bound Communication Complexity for k-CNF

Entry ID: 1a7e97ca7d97 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-29 02:34:02 UTC

1. Conjecture

Field A (mathematical branch): Algebra (Quaternion Algebras) **Field B** (complexity object): Communication Complexity (k-CNF)

Statement:

{‘sentence_1’: ‘For every instance of a k-CNF satisfiability problem with n variables, the minimal exponent of the quaternion algebra associated with its clause-formula is upper bounded by $O(n^2)$.’, ‘sentence_2’: ‘That is, if A is the quaternion algebra generated by the coefficients of the clauses in the k-CNF formula F , then $\exp(A) \leq c \cdot n^2$ for some constant c .’, ‘sentence_3’: ‘The communication complexity of solving F is at least logarithmic in this exponent.’}

Rationale (proposer’s reasoning):

{‘sentence_1’: ‘Quaternion algebras have been studied in algebraic number theory but are rarely applied to communication complexity, which deals with the distribution of information.’, ‘sentence_2’: ‘This conjecture suggests a potential link between abstract algebra and the concrete task of solving k-CNF problems, offering new insights into the resources needed for computation.’, ‘sentence_3’: ‘If true, it would reveal a novel way to leverage algebraic structures in complexity theory.’}

Taxonomy category: communication_complexity_quaternion_algebras (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 8934c83be67b1e2d

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For the k-CNF satisfiability problem, if $\exp(A) \leq c \cdot n^2$ for all clauses in F and a majority of seeds s , where $\exp(A)$ is the minimal exponent of the quaternion algebra associated with clause-formula F , and c is a constant.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	1.00	SAFE	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "minimal exponent quaternion algebras" AND "communication complexity k-CNF" - "quaternion algebra clause-formula" AND "k-CNF satisfiability problem" - "exponent bound A communication complexity logarithmic"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_k_cnf(n, k):
        variables = set(range(1, n + 1))
        clauses = []
        for _ in range(k):
            clause = random.sample(variables, random.randint(1, n))
            if random.choice([True, False]):
                clause = [-x for x in clause]
            clauses.append(clause)
        return clauses

    def construct_quaternion_algebra(clauses):
        # Simplified version of constructing a quaternion algebra
        # This is a placeholder and does not reflect actual quaternion algebra construction
        return sum(len(clause) ** 2 for clause in clauses), len(clauses)

    def communication_complexity(exponent):
        return math.log(exponent, 2)

    n = random.randint(5, 40)
    k = random.randint(n // 2, n * 2)
    clauses = generate_k_cnf(n, k)
    exponent, num_clauses = construct_quaternion_algebra(clauses)
    comm_complexity = communication_complexity(exponent)
```

```

return {
    "metric_name": "communication_complexity",
    "metric_value": comm_complexity,
    "instances_tested": 1,
    "conjecture_holds": comm_complexity >= math.log(n**2, 2),
    "counterexample": "" if comm_complexity >= math.log(n**2, 2) else f"n={n}, k={k}"
}

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [random.getrandbits(32) for _ in range(30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        counterexample = results[first_failing_seed]["counterexample"]
        print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_bdfa9e7b.py", line 58, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_bdfa9e7b.py", line 41, in run_trial
    clauses = generate_k_cnf(n, k)
              ^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_bdfa9e7b.py", line 25, in generate_k_cnf
    clause = random.sample(variables, random.randint(1, n))
              ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/usr/lib/python3.12/random.py", line 413, in sample
    raise TypeError("Population must be a sequence. ")
TypeError: Population must be a sequence. For dicts or sets, use sorted(d).

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means we cannot verify the conjecture's conditions. | next: Re-run the test with proper error handling to ensure it completes and produces results.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13001
2	preregistration	ollama_remote	glm4:latest	0	0	5731
3	novelty	ollama_remote	glm4:latest	0	0	4660
4	novelty	ollama_remote	glm4:latest	0	0	5745
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15695
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9076
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8099
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7355
9	judge	ollama_remote	glm4:latest	0	0	8524

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 77886 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/1a7e97ca7d97.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/1a7e97ca7d97.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/1a7e97ca7d97.tar.gz (if generated)